

Layered Bitemporal Graphs: Statement Identity, Two Clocks, and Metagraphs as Tags

Volodymyr Pavlyshyn

Independent researcher

Draft preprint, October 3, 2026

Abstract

Memory for software agents has to say more than *what* is true. It has to say how sure it is, where a fact came from, when the fact held in the world, when the system learned it, and what it believed before a correction. We study *layered bitemporal graphs*: graphs in which every statement (s, p, o) has its own identity, any statement can be the subject or object of another (so annotations stack into layers of unbounded depth), and every statement carries both a transaction-time lifetime and a valid-time interval. Retraction never deletes. It closes a lifetime and cascades to everything that rests on the retracted statement.

We give the model a small order-theoretic core. A set of statements is *grounded* when every statement in it finds its referents in it. The largest grounded subset of any set, its *ground core* γ , is an interior operator of an Alexandrov topology. From this single operator we derive: (i) the retraction cascade is exactly the complement of $\gamma(X \setminus \{e\})$ and is the unique least retraction that keeps memory grounded; (ii) every transaction-time snapshot is grounded if and only if lifetimes nest along references, which an engine guarantees under a live-reference discipline; (iii) valid time does not nest by itself, and the largest grounded valid-time slice is given by an *effective validity* with a closed form; (iv) correcting a fact by cascade-and-replay is an isomorphism on the ground core of its cascade set. We then show that metagraphs (edges that are vertices, and vertices or edges that contain subgraphs) embed faithfully into layered graphs using membership statements whose graph name may itself be a statement, and we prove a *Bitemporal Lifting Theorem*: for every well-formed temporal metagraph, taking a bitemporal snapshot commutes with the encoding. Metagraphs therefore inherit time travel, valid-time slicing and cascade semantics without any temporal machinery of their own. The model is implemented in Tiramemsu, an embedded graph database on SQLite. Formalising it surfaced three places where the implementation admitted ungrounded states, and we give corrections derived from the same operator.

1 Introduction

Consider an agent that reads in a chat log that Alice works at Acme. It records the fact, notes that it is 80% confident, that the source was a particular conversation, and that a belief it holds elsewhere is supported by this fact. A month later, Alice says she actually joined in February, not January. A week after that, an auditor asks what the agent believed on the 3rd of March, on what evidence, and since when.

Each part of this scenario has a mature answer in some literature. Annotating a fact needs statement-level identity, as in reification, RDF-star and RDF 1.2 [21, 22], singleton properties [36], Wikidata qualifiers [23], the Amazon Neptune OneGraph model [33] and the domain graph of MillenniumDB [46]. Distinguishing *when it was true* from *when we knew* is the bitemporal model of temporal databases [28, 29, 42, 43], standardised in SQL:2011 [32]. Keeping every past state queryable is the immutable-log design of Datomic [10] and XTDB [49]. Withdrawing everything that rests on a withdrawn premise is the job of truth maintenance systems [11, 13]. Grouping facts into larger units that can themselves be related is the metagraph [3, 8, 15, 17].

What is less studied is what happens when all of these live in *one* structure, at the granularity of a single statement. Then annotations have lifetimes, corrections must carry annotations along, a valid-time slice can contain a confidence score whose fact is not valid at that instant, and a container can be retracted while its contents stay. This paper gives that combined structure a precise semantics and proves the properties that make it safe to build on.

Contributions.

1. A formal model of *layered bitemporal graphs* (LBGs): statements with identity, a reference relation between statements, transaction-time lifetimes and valid-time intervals (Sections 3 and 4).
2. A single operator, the *ground core* γ , and a lemma that identifies grounded sets with the open sets of an Alexandrov topology (Lemma 1).
3. Theorems on the operations: stability of time travel (Theorem 3), minimality of the retraction cascade (Theorem 4), groundedness of snapshots as lifetime nesting (Theorem 5), effective validity (Theorem 7), and correctness of correction by cascade-and-replay (Theorem 8) (Section 5).
4. A faithful encoding of temporal metagraphs into LBGs and the *Bitemporal Lifting Theorem* (Theorem 10), with corollaries for unfold, nesting, fold and container retraction (Section 6).
5. A realisation in an embedded database and a conformance analysis: three ungrounded states the implementation admitted, each found by the theory and confirmed by a test, with corrections that are again instances of γ (Section 8).

Terminology. *Layered* here means stratified by reference: a statement about a statement sits one layer above it. This is not the *multilayer network* of network science [31], where layers are parallel edge sets over a shared vertex set. It is close to the *multilayer graphs* of Angles et al. [2], in which edges have identifiers and can be endpoints of other edges; we add the two clocks and the operations that keep such structures consistent over time.

2 Background and Related Work

Bitemporal data. Snodgrass and Ahn separated *valid time*, when a fact holds in the modelled world, from *transaction time*, when it is current in the database [43]. The Bitemporal Conceptual Data Model attaches to each tuple a set of bitemporal chronons and shows that several representational models are snapshot-equivalent views of it [28]. The consensus glossary fixes the vocabulary [29], and SQL:2011 introduced application-time and system-versioned tables [32]. Our transaction-time lifetimes are exactly system versioning, and our valid-time intervals are application time. What is new is that the tuples refer to one another, so a snapshot can be *ungrounded*, a question that never arises for flat relations.

Temporal graphs. Network science studies graphs whose edges are active at times, and time-respecting paths along them [7, 25, 30]. Wu et al. give efficient algorithms for earliest-arrival, latest-departure, fastest and shortest temporal paths [48]. In databases, temporal property-graph models attach intervals to nodes and edges [12, 41] and temporal RDF attaches them to triples [20]. These models version vertices and edges. They do not let a timestamped edge be the endpoint of another timestamped edge, and so they never meet the interaction between nesting and time that is our subject. Systems work on temporal graph stores, such as AeonG [26], Clock-G [34] and ChronoGraph [5], optimises the storage and traversal of versioned vertices and

edges, and temporal graph algebra [35] gives point- and interval-based semantics for evolving graphs.

Statement-level identity. RDF reification, RDF-star [21] and the RDF 1.2 reifier model [22] let a triple be talked about. Singleton properties mint a predicate per occurrence [36]. Hernández, Hogan and Krötzsch compared these encodings for Wikidata’s qualified statements [23, 45], and benchmarks show the cost of encoding statement metadata on top of plain triples [14, 37]. Named graphs group triples under a name that can carry provenance [6]. OneGraph [33] and MillenniumDB’s domain graph [46] make every edge an object with an identifier. Gelling, Fletcher and Schmidt’s *statement graphs* [16] make statement identity the unifying primitive of RDF, RDF-star and property graphs, and are the closest data model to ours; Angles et al.’s multilayer graphs [2] are another. Neither has time, retraction or containment. We adopt this approach in its strongest form: each *occurrence* of a statement has an identifier, properties included. Unlike RDF 1.2, where a reifier may stand for several triples and a triple for several reifiers, our identifier is a total function to content.

Metagraphs and hypergraphs. Basu and Blanning define a metagraph over a generating set with edges between *sets* of elements [3]. Chernenkiy, Gapanyuk and colleagues add metaverices and metaedges that contain fragments, and annotations that are themselves metagraphs [8, 15]. Goertzel studies *folding* a subgraph into a vertex and *unfolding* it again [17]. HypergraphDB stores hyperedges that can point to hyperedges [27]. The use of metagraphs for agent memory is developed in [39]. None of these treats time. Our contribution here is to show that time need not be added to the metagraph at all, because it lifts from the statements underneath (Theorem 10).

Truth maintenance and belief revision. A justification-based TMS withdraws every belief whose justification loses support [13]; an assumption-based TMS tracks the assumption sets under which beliefs hold [11]. AGM belief revision axiomatises contraction and revision of theories [1]. Our cascade is a syntactic, monotone cousin of TMS withdrawal. Support is structural (a layer rests on its base), there are no alternative justifications, and nothing is forgotten, because withdrawal closes a lifetime. Theorem 4 plays the role of a minimal-change postulate: the cascade withdraws exactly what it must and nothing more.

Provenance. Why- and where-provenance [4] and provenance semirings [18] annotate query answers with the source tuples that produced them. Statement identity makes such citations durable, because a cited statement can later be checked for retraction in any snapshot.

Agent memory. Recent agent-memory systems store temporal knowledge graphs whose edges carry validity windows and system times; Zep’s Graphiti is a prominent example [40]. Other memory layers manage what enters a bounded context [38], extract and consolidate memories [9, 50], or retrieve over a knowledge graph [19]; benchmarks such as LongMemEval test knowledge updates and temporal reasoning explicitly [47]. None of these defines what a correction does to the annotations on a fact. The surveys of knowledge graphs [24] summarise the broader design space. Our model can be read as giving such systems a precise account of what a correction, a retraction or a snapshot does to the annotations on a fact.

3 Layered Graphs

3.1 Statements and references

Fix disjoint countable sets: nodes $\mathcal{N}$, literals $\mathcal{L}$, statement identifiers $\mathcal{E}$ and transaction numbers $\mathbb{T} = \{1, 2, \dots\}$. Predicates are nodes, $\mathcal{P} \subseteq \mathcal{N}$.

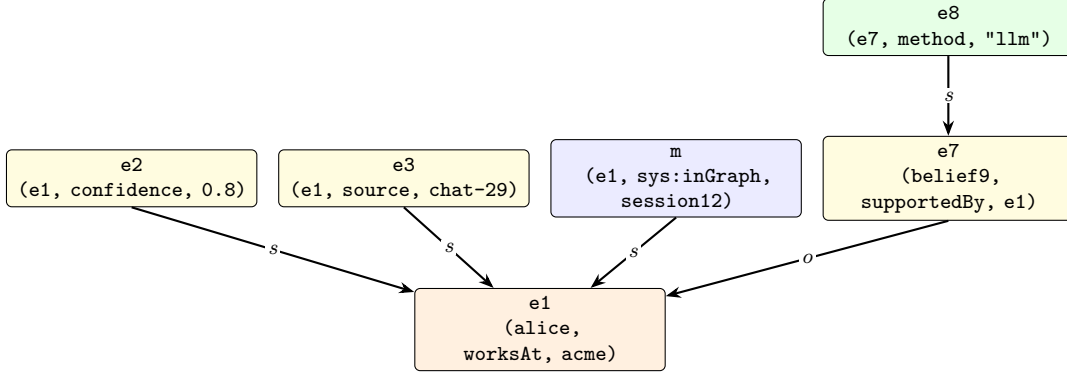


Figure 1: A layered graph. Rows are ranks 0, 1 and 2. Arrows are the reference relation $\rightarrow$, labelled by the position (subject or object) that holds the referenced statement. The support of **e8** is $\{\mathbf{e8}, \mathbf{e7}, \mathbf{e1}\}$; the dependents of **e1** are all six statements. The membership **m** places **e1** in a named graph, and is itself a statement.

Definition 1 (Layered graph). A *layered graph* is a finite set $S \subseteq \mathcal{E}$ of statements together with a content function $\kappa: S \rightarrow (\mathcal{N} \cup \mathcal{E} \cup \mathbb{T}) \times \mathcal{P} \times (\mathcal{N} \cup \mathcal{L} \cup \mathcal{E} \cup \mathbb{T})$, written $\kappa(e) = (s_e, p_e, o_e)$, such that no statement mentions itself: $s_e \neq e \neq o_e$.

The identifier e is the identity of one *occurrence* of a fact. Two statements may have equal content (parallel edges); content is immutable. A statement whose object is a literal is a property; one whose subject or object is a statement is a layer or a citation.

Definition 2 (References, support, dependents). For $e, e' \in S$ write $e \rightarrow e'$ if $e' \in \{s_e, o_e\}$. Let $\rightarrow^*$ be the reflexive-transitive closure. The *support* of e is $\downarrow e = \{e' \in S : e \rightarrow^* e'\}$, everything e rests on, e included. For $X \subseteq S$ the *dependents* of e in X are $\uparrow_X e = \{x \in X : x \rightarrow^* e\}$.

The relation $\rightarrow$ may contain cycles of length at least two (statement a about b about a). Proposition 6 shows that cycles cannot arise under the live-reference discipline of Section 5. Where $\rightarrow$ is acyclic the *rank* of a statement, 0 if it references no statement and otherwise one more than the largest rank it references, is the layer it sits in.

3.2 Grounded sets and the ground core

The central notion of this paper is the following.

Definition 3 (Grounded set, ground core). A set $X \subseteq S$ is *grounded* if $\downarrow e \subseteq X$ for every $e \in X$, that is, if $e \in X$ and $e \rightarrow e'$ imply $e' \in X$. The *ground core* of an arbitrary $X \subseteq S$ is

$$\gamma(X) = \{e \in X : \downarrow e \subseteq X\}.$$

A grounded set never contains an annotation without the statement it annotates, nor a citation without what it cites.

Lemma 1 (Ground core). For all $X, Y \subseteq S$ and families $(X_i)_{i \in I}$:

1. $\gamma(X) \subseteq X$, $\gamma(\gamma(X)) = \gamma(X)$, and $X \subseteq Y$ implies $\gamma(X) \subseteq \gamma(Y)$;
2. $\gamma(X)$ is grounded, and it is the largest grounded subset of X ;
3. $\gamma(\bigcap_i X_i) = \bigcap_i \gamma(X_i)$;
4. arbitrary unions and intersections of grounded sets are grounded.

Hence the grounded sets are the open sets of an Alexandrov topology on S , and γ is its interior operator.

Proof. (1) Deflation and monotonicity are immediate. If $e \in \gamma(X)$ and $e' \in \downarrow e$ then $\downarrow e' \subseteq \downarrow e \subseteq X$ by transitivity, so $\downarrow e \subseteq \gamma(X)$; this shows $e \in \gamma(\gamma(X))$ and also that $\gamma(X)$ is grounded. (2) If $Y \subseteq X$ is grounded and $e \in Y$, then $\downarrow e \subseteq Y \subseteq X$, so $e \in \gamma(X)$. (3) $\downarrow e \subseteq \bigcap_i X_i$ iff $\downarrow e \subseteq X_i$ for all i . (4) The defining condition is preserved by arbitrary unions and intersections. A topology closed under arbitrary intersections is Alexandrov; its open sets here are the down-sets of the preorder $\rightarrow^*$. $\square$

Lemma 2 (Cascade as a core). *If X is grounded and $e \in X$, then $X \upharpoonright_{\uparrow_X e} = \gamma(X \setminus \{e\})$.*

Proof. Let $x \in X \setminus \{e\}$. Because X is grounded, $\downarrow x \subseteq X$, so $\downarrow x \subseteq X \setminus \{e\}$ iff $e \notin \downarrow x$ iff $x \notin \uparrow_X e$. The statement e itself lies in $\uparrow_X e$ and not in $X \setminus \{e\}$. $\square$

4 Two Clocks

4.1 Lifetimes and intervals

Transaction numbers are gap-free and strictly increasing, one per committed transaction. Valid-time instants $\mathbb{V}$ form a total order (epoch milliseconds in the implementation); $\pm\infty$ denote unbounded ends. All intervals are half-open.

Definition 4 (Layered bitemporal graph). A *layered bitemporal graph* (LBG) is a layered graph (S, κ) with two further functions on S : a *lifetime* $\lambda(e) = [t_{\text{add}}(e), t_{\text{ret}}(e))$ with $t_{\text{add}}(e) \in \mathbb{T}$ and $t_{\text{ret}}(e) \in \mathbb{T} \cup \{\infty\}$, $t_{\text{add}}(e) \leq t_{\text{ret}}(e)$; and a *valid interval* $\nu(e) = [v_{\text{from}}(e), v_{\text{to}}(e)) \subseteq \mathbb{V}$, non-empty. Content $(\kappa(e), \nu(e))$ is immutable; only $t_{\text{ret}}(e)$ may change, once, from ∞ to a transaction number.

A statement with $t_{\text{add}}(e) = t_{\text{ret}}(e)$ was asserted and retracted inside one transaction and is visible in no snapshot. Changing a valid interval is a correction (Section 5.5), never an update.

Definition 5 (Views). For $t \in \mathbb{T}$ and $v \in \mathbb{V}$:

$$V_t = \{e \in S : t \in \lambda(e)\}, \quad V^v = \{e \in S : v \in \nu(e)\}, \quad V_t^v = V_t \cap V^v.$$

V_t is the *as-of* view, V^v the *valid-at* slice, and V_t^v the bitemporal snapshot. The *history* view is S itself.

A query evaluated on a view sees the content of exactly the statements in the view. The implementation lets each pattern of a query choose its own view; here one view per query is enough.

4.2 Histories and stability

A database evolves by committed transactions. Let $S^{(t)}$ denote the LBG after transaction t , with $t_{\text{ret}} = \infty$ for statements still live. A transaction t may only (a) insert statements with $t_{\text{add}} = t$ and fresh identifiers, and (b) set $t_{\text{ret}}(e) = t$ for statements live after $t - 1$. This is the *never-forget* discipline: no row is deleted and no identifier is reused. The implementation enforces it with database triggers, so it holds for any client of the file, not only the engine.

Theorem 3 (Stability of time travel). *For all $t \leq t' \leq t''$, the view V_t computed in $S^{(t')}$ equals V_t computed in $S^{(t'')}$, statement for statement and with equal content. Moreover, V_t equals the set obtained by replaying, in order, the insert and retract events with transaction number at most t .*

Proof. Every transaction $u \in (t', t'']$ inserts statements with $t_{\text{add}} = u > t$, which are not in V_t , and sets $t_{\text{ret}}(e) = u > t$ on statements that were live, which keeps $t \in \lambda(e)$ exactly when it held before. Content never changes. For the second claim, a statement is in the replayed set iff its insert event has time $\leq t$ and its retract event, if any, has time $> t$, which is the definition of V_t . $\square$

The theorem is what makes an answer *about the past* reproducible: a snapshot never changes once its transaction has committed. The second half is checked in the implementation by property-based tests over random operation sequences.

5 Operations and Their Guarantees

5.1 Assert and create

$\text{Assert}(s, p, o, \nu)$ is idempotent: if a live statement with the same content and an overlapping valid interval exists, it is returned unchanged; otherwise a fresh statement is inserted. Create always inserts, which is how parallel edges are made. Both insert statements whose subject or object may be another statement. What they are allowed to reference decides whether snapshots stay grounded.

Definition 6 (Live-reference discipline). Let $\Lambda \subseteq \mathcal{P}$ be a set of *lineage* predicates. A transaction satisfies the *live-reference discipline* (LR) if every statement e it inserts with $p_e \notin \Lambda$ references only statements that are live at the moment of insertion (inserted earlier and not retracted).

Write $\rightarrow_\Lambda$ for the restriction of $\rightarrow$ to statements whose predicate is not in Λ , and call a set *structurally grounded* when it is grounded for $\rightarrow_\Lambda$. Lineage links, such as the link from a corrected fact to its predecessor, point into the past by design and are exempt.

5.2 Retraction and its cascade

Retracting e retracts, in the same transaction, e and every live statement that transitively references it through a subject or object position: its layers, the statements that cite it, the layers on those, and so on. Nodes never cascade.

Theorem 4 (Cascade minimality). *Let X be the live set before a retraction of $e \in X$, and let X be grounded. The cascade retracts exactly $\uparrow_X e$, and:*

1. $X \setminus \uparrow_X e = \gamma(X \setminus \{e\})$, so the remaining live set is grounded;
2. $\uparrow_X e$ is the least set R such that $e \in R \subseteq X$ and $X \setminus R$ is grounded.

Proof. The cascade is a breadth-first search from e over inverse references among live statements, with a visited set, so it terminates on cycles and reaches exactly $\uparrow_X e$. Claim 1 is Lemma 2. For Claim 2, let R be any such set. Then $X \setminus R$ is a grounded subset of $X \setminus \{e\}$, hence contained in its core $X \setminus \uparrow_X e$ by Lemma 1(2), so $\uparrow_X e \subseteq R$. $\square$

The theorem says that the cascade is forced. Any retraction that leaves memory grounded must take at least $\uparrow_X e$ with it, and the cascade takes nothing else. In belief-revision terms it is the minimal contraction for the structural support relation. The same set can be computed without the writer, as a read on any view: as-of t it shows what depended on e then. It is also the set of ends of the regular path $(\text{sys:subject} \mid \text{sys:object})^*$ from e . Both equivalences are checked by property tests in the implementation.

5.3 Snapshots are grounded exactly when lifetimes nest

Theorem 5 (Transaction-time nesting). *For a history S the following are equivalent:*

1. *every as-of view V_t is structurally grounded;*
2. *for every structural reference $e \rightarrow_\Lambda e'$, $\lambda(e) \subseteq \lambda(e')$.*

If every transaction satisfies LR, every retraction cascades, and every correction replays a grounded set (Theorem 8), then both hold.

Proof. (1) $\Leftrightarrow$ (2): V_t is grounded for every t iff for each reference $e \rightarrow_\Lambda e'$ and each t , $t \in \lambda(e)$ implies $t \in \lambda(e')$, which is $\lambda(e) \subseteq \lambda(e')$. For the sufficient condition we show by induction over the operations of each transaction that the live set stays structurally grounded; the committed live set after t is V_t , because a statement inserted and retracted within t has an empty lifetime. Insertion under LR adds a statement whose structural referents are live. Retraction preserves groundedness by Theorem 4, applied to $\rightarrow_\Lambda$ -dependents (the implementation's cascade follows all references, lineage included; the remaining set is still structurally grounded, because if $x \rightarrow_\Lambda y$ and y is removed then $x \rightarrow y \rightarrow^* e$, so x is removed too). Correction is covered by Theorem 8. $\square$

Condition 2 gives a geometric reading of a grounded history. Draw each lifetime as a segment on the transaction axis; a layer's segment lies inside the segment of the statement it rests on. Lineage links are the opposite case: their segment begins exactly where the referenced statement's segment ends.

Example 1 (Why LR is needed). Assert $e_1 = (\text{alice}, \text{worksAt}, \text{acme})$ at $t = 1$, retract it at $t = 2$, and at $t = 3$ assert $e_2 = (e_1, \text{confidence}, 0.8)$. Then $e_2 \in V_3$ but $e_1 \notin V_3$: the as-of view at 3 holds a confidence about nothing. Condition 2 fails, since $\lambda(e_2) = [3, \infty) \not\subseteq [1, 2) = \lambda(e_1)$.

Proposition 6 (Acyclicity). *Suppose every inserted statement references only statements that already exist, and that statements with a lineage predicate are inserted only by corrections as in Theorem 8. Then the reference relation of the whole history, retracted statements and lineage links included, is acyclic, and rank is well defined. In particular this holds under LR, which implies the first condition for all non-lineage statements.*

Proof. By induction on insertions. A statement inserted outside a correction references statements that already exist, and no existing statement references it, because content is immutable and identifiers are fresh; so it closes no cycle. A correction inserts a copy $\sigma(C')$ of a set C' of existing statements, with references inside C' mapped by the bijection σ and references leaving C' kept. No earlier statement references a copy. A cycle through a copy would therefore lie entirely inside $\sigma(C')$, and its preimage would be a cycle in C' , contradicting the induction hypothesis. The lineage link references two existing statements and is referenced by none. $\square$

Cycles are therefore a symptom of a reference to a statement that did not yet exist. Under acyclicity, the cascade of e contains only statements of strictly higher rank than e (apart from e): retraction propagates upward through the layers and never sideways.

5.4 Valid time does not nest, and how to read it

Valid intervals are content chosen by the writer, and a layer's interval is independent of its base's. This is deliberate: a confidence assessed in 2026 about a fact that held in 2020 is valid in 2026. Consequently V^v , and with it V_t^v , need not be grounded even when every V_t is. A reader who wants a grounded slice can take its core, and the core has a closed form.

Definition 7 (Effective validity). For a view X and $e \in X$ let $\nu_X^*(e) = \bigcap \{\nu(e') : e' \in \downarrow e\}$ if $\downarrow e \subseteq X$, and $\emptyset$ otherwise.

An intersection of half-open intervals is a half-open interval, possibly empty, so effective validity is again an interval and can be stored or indexed like any other.

Theorem 7 (Effective validity). *For every view X and instant v :*

1. $\gamma(X \cap V^v) = \{e \in X : v \in \nu_X^*(e)\}$, the largest grounded subset of the slice;
2. ν_X^* is the greatest fixpoint, under pointwise inclusion, of $F(\varphi)(e) = \nu(e) \cap \bigcap_{e \rightarrow e'} \varphi(e')$ on $\gamma(X)$.

Proof. (1) $e \in \gamma(X \cap V^v)$ iff every $e' \in \downarrow e$ lies in X and has $v \in \nu(e')$, iff $v \in \nu_X^*(e)$. Maximality is Lemma 1(2). (2) Because $\downarrow e = \{e\} \cup \bigcup_{e \rightarrow e'} \downarrow e'$, ν_X^* is a fixpoint. If φ is any fixpoint then $\varphi(e) \subseteq \nu(e)$ and $\varphi(e) \subseteq \varphi(e')$ whenever $e \rightarrow e'$, so following any reference path from e to e' and chaining these inclusions, $\varphi(e) \subseteq \nu(e')$ for every $e' \in \downarrow e$ (cycles are harmless, since only finite paths are used), that is $\varphi \subseteq \nu_X^*$. The fixpoint also exists by Tarski's theorem [44], since F is monotone on a complete lattice; the closed form makes the theorem constructive and valid in the presence of cycles. $\square$

Theorems 5 and 7 show an asymmetry between the clocks. Transaction time nests by construction, because the engine controls it. Valid time is the writer's claim about the world and nests only if the reader asks for it, through γ . The asymmetry is a design choice, and the closed form makes the reader's side cheap: ν^* is an interval intersection along the support, computable during the same traversal that evaluates a layered pattern.

5.5 Correction by cascade-and-replay

A correction changes the object or valid interval of a live statement r while keeping everything said about it. It computes the cascade set $C = \uparrow_X r$, retracts it, and inserts a copy of it under fresh identifiers with the patch applied to the root, then records a lineage link $(\sigma(r), \text{sys:supersedes}, r)$ with $\text{sys:supersedes} \in \Lambda$.

Graph memberships need care. A membership $(x, \text{sys:inGraph}, g)$ with $x \in C$ and $g \notin C$ says that the old statement belonged to a graph. The writer of the correction should decide whether the new statement belongs too, so such *foreign memberships* $F \subseteq C$ are dropped. A membership whose graph is itself in C , the contents of an edge that is a container (Section 6), follows the edge.

Theorem 8 (Correction). *Let the live set X be structurally grounded, $r \in X$, $C = \uparrow_X r$, F the foreign memberships in C , and $R = \gamma_C(C \setminus F)$, where $\gamma_C(Y) = \{e \in Y : \downarrow e \cap C \subseteq Y\}$ is the ground core relative to C , which only asks references that land inside C to stay in Y . So R holds the statements of C whose support within C avoids F . Replaying R under a fresh bijection σ yields the live set $X' = (X \setminus C) \cup \sigma(R) \cup \{\ell\}$ with ℓ the lineage link, and:*

1. σ is an isomorphism of the reference structure from R onto $\sigma(R)$: for $x, y \in R$, $x \rightarrow y$ iff $\sigma x \rightarrow \sigma y$; content and valid time are preserved except at the root, and references from R to $X \setminus C$ are preserved verbatim;
2. X' is structurally grounded;
3. the pre-correction structure is preserved: $C \subseteq V_{t-1}$, unchanged forever by Theorem 3.

If R is replaced by $C \setminus F$, Claim 2 holds iff no statement of $C \setminus F$ references a statement of F .

Proof. (1) holds by construction: each copy has its subject and object rewritten through σ where they lie in R and kept otherwise. (2) $X \setminus C$ is grounded by Theorem 4. Take $x \in R$ and $x \rightarrow y$. If $y \in C$, then $y \in \downarrow x$ within C , so $y \notin F$ and $y \in R$ by definition of the core, and $\sigma x \rightarrow \sigma y \in X'$. If $y \notin C$, then $y \in X$ because X is grounded, and $y \in X \setminus C \subseteq X'$. The link ℓ is lineage and is exempt. (3) Every member of C was live after $t - 1$. For the last sentence: replaying $x \in C \setminus F$ with $x \rightarrow m \in F$ yields a copy that references $\sigma(m)$, an identifier that names no statement. $\square$

A correction that is retracted by a cardinality-one rule on the new root cascades again, and Theorem 4 applies. Because the old cascade set includes earlier lineage links, the lineage of a fact after n corrections is a path of length n through `sys:supersedes`.

6 Metagraphs on Layered Graphs

A metagraph has two powers that a graph lacks [3, 8, 15, 39]. *Edges are vertices*: a relationship can be an endpoint of another relationship. *Vertices are containers*: a vertex, or an edge, can hold a subgraph. Statement identity gives the first power directly. This section shows that membership statements whose graph name may be a statement give the second, and that time then comes for free.

6.1 Temporal metagraphs

Definition 8 (Metagraph). A *metagraph* is a tuple $M = (V, E, \text{src}, \text{tgt}, \text{lab}, \mu, \alpha)$ where V and E are disjoint finite sets of vertices and edges, $A = V \cup E$ is the set of *atoms*, $\text{src}, \text{tgt}: E \rightarrow 2^A \setminus \{\emptyset\}$ give each edge non-empty source and target sets with $x \notin \text{src}(x) \cup \text{tgt}(x)$, $\text{lab}: E \rightarrow \mathcal{P}$ labels edges, μ is a finite set of *memberships*, each with a member $\text{mem}(m) \in A$ and a container $\text{con}(m) \in A$, $\text{mem}(m) \neq \text{con}(m)$, and α is a finite set of *annotations*, each with a bearer $\text{bear}(a) \in A$, a key in $\mathcal{P}$ and a literal value.

The definition covers the classical variants. Basu–Blanning metagraphs are the case $\mu = \alpha = \emptyset$ with $\text{src}, \text{tgt} \subseteq V$; edges whose endpoints are edges give edges-as-vertices; a vertex with members is a metavertex; an edge with members is a metaedge in the sense of Gapanyuk; annotations give attributes. Hypergraphs take $\text{src} = \text{tgt}$. Atoms, memberships and annotations are collectively the *components* of M .

Each component c *depends on* the atoms it mentions: an edge on its endpoints, a membership on its member and container, an annotation on its bearer. A vertex depends on nothing.

Definition 9 (Temporal metagraph). A *temporal metagraph* is a metagraph M whose components each carry a lifetime $\lambda(c)$ and a valid interval $\nu(c)$. Its snapshot $H(t, v)$ keeps the components c with $t \in \lambda(c)$ and $v \in \nu(c)$. It is *well-formed* if, whenever c depends on d , $\lambda(c) \subseteq \lambda(d)$ and $\nu(c) \subseteq \nu(d)$.

Components are occurrences, like statements: re-adding a removed membership creates a new membership with its own lifetime. Well-formedness says that no component outlives what it mentions on either clock. It is precisely what makes every snapshot $H(t, v)$ a metagraph, since a snapshot of an ill-formed history can contain an edge whose endpoint is absent.

6.2 The encoding

Fix fresh predicates `mg:Vertex`, `mg:src`, `mg:tgt` and the membership predicate `sys:inGraph`. The encoding assigns each atom x an *anchor* statement $h(x)$, its handle, and encodes every other component as a layer on handles.

Definition 10 (Encoding). Given a temporal metagraph, `enc` produces the LBG with the following statements, each inheriting the lifetime and valid interval of the component it encodes:

- for a vertex x : $h(x) = (n_x, \text{rdf:type}, \text{mg:Vertex})$;
- for an edge x : the anchor $h(x) = (n_x, \text{rdf:type}, \text{lab}(x))$ and the role statements
 $(h(x), \text{mg:src}, h(y))$ for $y \in \text{src}(x)$, $(h(x), \text{mg:tgt}, h(y))$ for $y \in \text{tgt}(x)$;
- for a membership m : $(h(\text{mem}(m)), \text{sys:inGraph}, h(\text{con}(m)))$;

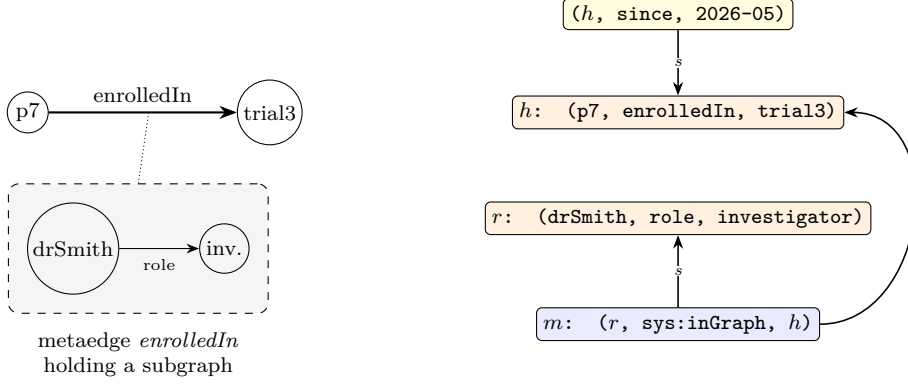


Figure 2: A metaedge holding a subgraph, and its encoding with the compact binary form of Remark 1. Retracting h cascades to the membership m and the annotation, and leaves the contained statement r live: the container goes, its contents stay (Corollary 12).

- for an annotation a with key k and value d : $(h(\text{bear}(a)), k, d)$.

Here n_x is a fresh node per atom. Role statements inherit the edge’s lifetime and interval.

The membership statement is the shape introduced by statement graphs in the implementation: its object, the graph name, is the identifier of a statement, so a container can be a vertex or an edge alike. Membership is a *tag* on the member, not a copy, so an atom in several containers keeps one identity and one set of annotations.

Definition 11 (Decoding). **dec** maps an LBG back to a metagraph by reading the four statement shapes: anchors with **mg:Vertex** give vertices; other anchors with role statements give edges whose endpoints are the atoms of the referenced anchors; **sys:inGraph** statements between anchors give memberships; remaining layers on anchors with literal objects give annotations.

Theorem 9 (Faithful encoding). *For every metagraph M , $\text{dec}(\text{enc}(M)) \cong M$, the isomorphism mapping each atom x to the atom decoded from $h(x)$. The image of **enc** consists of exactly the grounded LBGs in which every statement has one of the four shapes of Definition 10 (anchor, role, membership, or annotation with a literal object), role and membership statements have anchor subjects and anchor objects, and every edge anchor has at least one role of each kind.*

Proof. Each component of M produces statements of exactly one shape, and **dec** reads shapes back component by component, so $\text{dec} \circ \text{enc}$ is the identity up to naming of atoms. Conversely, in an LBG with these constraints every statement is read by exactly one clause of **dec**, so it decodes into a metagraph whose encoding is the LBG up to renaming of identifiers. Without the shape constraint a statement such as $(h(x), p, h(y))$ with an ordinary predicate p would be read by no clause. Groundedness is necessary because every role, membership and annotation references an anchor. $\square$

The subject-kind constraints are expressible as per-predicate subject-type declarations, which the implementation checks on every write (*typed layers*). A store can therefore refuse writes that would leave the image of the encoding.

Remark 1 (Compact binary edges). An edge with one source and one target can be encoded by the single statement $(h(y), \text{lab}(x), h(z))$, or $(n_y, \text{lab}(x), n_z)$ when vertex lifetimes are not needed. This is what users write in practice, and the theorems below hold for it unchanged. It has one operational advantage: the edge statement references its endpoints, so retracting an endpoint retracts the edge by cascade. In the general form only the role statements reference endpoints, so deleting a vertex at the metagraph level must also retract the anchors of the n -ary edges it belonged to.

6.3 The lifting theorem

Theorem 10 (Bitemporal Lifting). *Let H be a well-formed temporal metagraph. For every transaction time t and valid instant v :*

1. $\text{dec}(V_t^v(\text{enc}(H))) \cong H(t, v)$, that is, snapshot and encoding commute;
2. $V_t^v(\text{enc}(H))$ is grounded, so it equals its own core and no effective-validity correction is needed;
3. the history $\text{enc}(H)$ satisfies the nesting condition of Theorem 5, so it can be produced by transactions that obey LR.

If H is not well-formed but every edge is binary and encoded in the compact form of Remark 1, then $\text{dec}(\gamma(V_t^v(\text{enc}(H))))$ is the largest sub-metagraph of $H(t, v)$ in which every component's dependencies are present.

Proof. (1) enc acts component by component and every statement inherits its component's lifetime and interval, so $V_t^v(\text{enc}(H))$ is exactly the set of statements encoding the components alive at (t, v) . Well-formedness guarantees that the atoms these components depend on are alive too, so the result is $\text{enc}(H(t, v))$, and Theorem 9 gives the claim. (2) Every reference in $\text{enc}(H)$ goes from the encoding of a component to the anchor of an atom it depends on, and well-formedness makes both alive at (t, v) . (3) The same argument with only the lifetime half of well-formedness gives $\lambda(e) \subseteq \lambda(e')$ for each reference. For the last sentence, in the compact form the references of every statement are exactly the dependencies of its component, so by Lemma 1(2) the core keeps exactly the components whose dependencies are present. The general n-ary form breaks this: an anchor $(n_x, \text{rdf:type}, \text{lab}(x))$ references no statement, so it survives the core even when its endpoints are absent, and the core can then keep role statements that point at an edge which no longer decodes. $\square$

The theorem is short to prove because the encoding is built for it: each metagraph component is a statement, each dependency a reference. Its consequence is that a metagraph needs no temporal theory of its own. As-of views, valid-time slices, histories, cascades and corrections are defined once on statements and act on metagraphs through the encoding.

6.4 Consequences for metagraph operations

Corollary 11 (Unfold and nesting in time). *For a well-formed H , the members of a container c in $H(t, v)$ are the subjects of the **sys:inGraph** statements with object $h(c)$ in $V_t^v(\text{enc}(H))$, and the transitive closure of membership at (t, v) is the answer of a regular path query over $\text{enc}(H)$ evaluated in that view.*

Corollary 12 (Containers retract, contents stay). *Retracting the anchor $h(c)$ of a container retracts every membership into c , every annotation on c and every statement resting on those, and retracts a member only if the member itself references c .*

Proof. By Theorem 4, the cascade is $\uparrow_X h(c)$. A membership into c has $h(c)$ as its object and so lies in it. A member's anchor lies in it only if it reaches $h(c)$ through references, which a membership, pointing from member to container, does not provide. $\square$

Corollary 13 (Fold is a non-destructive write). *Folding a set F of atoms into a container c at transaction t inserts one membership per atom of F . It changes no view $V_{t'}$ with $t' < t$, and for every $t'' \geq t$ before any later change and every v in the intersection of the intervals involved, unfolding c at (t'', v) returns F together with any earlier members.*

Proof. By Theorem 3 and Corollary 11. $\square$

Proposition 14 (Size). $|\text{enc}(M)| = |V| + |E| + \sum_{x \in E} (|\text{src}(x)| + |\text{tgt}(x)|) + |\mu| + |\alpha|$, linear in the size of M ; with compact binary edges, a binary edge costs one statement.

Correction interacts well with containers. A container edge’s memberships are in its cascade set and their graph is in it too, so a correction of the edge replays them onto the new edge (Theorem 8): a corrected enrolment keeps its investigators. The members keep their identifiers.

7 Paths Through Time

Layered graphs make paths across layers natural: a walk can go from a belief to the fact it rests on, to the source of that fact, through virtual hops from a statement to its subject or object. On top of the two clocks, a path can also be required to respect valid time. A walk is *time-respecting* from start time τ_0 if it can traverse each statement at some instant inside its valid interval, never going back [25, 30]. With τ the walk’s current time, a hop over interval $[a, b)$ is allowed iff $b > \tau$, and moves τ to $\max(\tau, a)$.

Proposition 15 (Earliest arrival). *The hop rule is monotone: if $\tau \leq \tau'$ and a hop is allowed at τ' , it is allowed at τ and ends no later. Hence a label-correcting search that keeps, for each pair of node and automaton state, the earliest time seen, and expands a pair again only on strict improvement, returns for each reachable end its earliest arrival over all time-respecting walks within the hop bound.*

This is the standard argument for earliest-arrival paths in temporal graphs [48], here over the product with a regular path automaton. In the implementation it is checked against brute-force enumeration of walks. By Theorem 10, the same search answers journey questions over a temporal metagraph.

8 Realisation and Conformance

8.1 Tiramemsu

The model is implemented in Tiramemsu, an embedded graph database written in Rust over SQLite, queried with SPARQL and Cypher over one store. A statement is one row $(eid, s, p, o, t_{\text{add}}, t_{\text{ret}}, v_{\text{from}}, v_{\text{to}}, \text{ret_kind})$. Every identifier is a 64-bit integer with a 4-bit type tag in the low bits, so statements, nodes, transactions and small literals share one column type and stay compact as SQLite varints. Permutation indexes are split into partial indexes over live rows, which serve the current view, and history indexes ordered by t_{add} descending, which serve as-of views with one range seek. Triggers reject any delete and any update other than setting t_{ret} once, so the never-forget discipline of Section 4 holds for any program that opens the file. Named graphs, including graphs named by statements, are membership rows in the same table; a graph costs one row and its index entries per member, and selecting a graph is a seek on the predicate–object index.

Several results of this paper are checked by property-based tests over random operation sequences and random layered graphs with reference cycles and memberships: replay equivalence (Theorem 3); equality of the read-only dependents, the dry-run cascade and the reverse path query (Theorem 4); round-tripping of fact bundles; and time-respecting search against enumeration (Section 7).

8.2 Conformance findings

Stating the theorems with explicit hypotheses made it possible to check whether the implementation meets them. At the time of writing it does not in three places, each confirmed by a test against the engine.

1. **References to retracted statements.** The write path checks self-reference but not liveness of a statement in subject or object position. Example 1 runs as written, as does a citation of a retracted statement, and the current view then contains a layer on a statement that is not in it. LR holds for confirmations and graph memberships, which do check liveness, but not for ordinary asserts.
2. **Forward references.** An identifier that has never been allocated is accepted as a subject. This is the only way, outside correction, to create a reference cycle (Proposition 6).
3. **Layers on dropped memberships.** A correction replays $C \setminus F$, not $\gamma_C(C \setminus F)$. If a foreign membership carries a layer (for instance who added the fact to a session), the layer’s copy references $\sigma(m)$, an identifier allocated but never inserted. The last sentence of Theorem 8 predicts exactly this.

The corrections follow from the theory. For (3), replay $\gamma_C(C \setminus F)$: the layers of a dropped membership are dropped with it. For (1) and (2), enforce LR on every user write, rejecting unallocated identifiers outright and retracted ones unless the predicate is declared as lineage. Declaring lineage predicates keeps a legitimate use: annotating a fact *after* it was retracted, for example to record why it was wrong, is a past-pointing reference by intent, and Theorem 5 exempts it as long as it is declared. Until LR is enforced, a reader can always obtain a grounded view as $\gamma(V_t)$, at the cost of a support traversal.

9 Discussion

Occurrences, not facts. Identity is per occurrence: two statements may have equal content, and an idempotent assert decides when a new occurrence is created. This is what lets a fact have two episodes with disjoint valid intervals, each with its own annotations. RDF 1.2’s many-to-many reifiers are more flexible; an LBG can model them by a reifier node with ordinary statements, losing the cascade from fact to annotation, which is the point of identity here.

Why valid time is not clamped. One could force a layer’s valid interval inside its base’s on write. We do not, because annotations are often claims *about* a fact made at another time. Theorem 7 gives readers the clamped semantics on demand. A schema flag that requests clamping for a given predicate (for example roles of an n-ary edge, whose validity is the edge’s) would make the reader-side core unnecessary for those predicates.

Branching and merging. The model has one linear transaction axis. Speculative transactions are evaluated and discarded, as in Datomic, and never enter the history. Merging the histories of several agents would need identifiers that are stable across stores; the implementation reserves bits for an origin, but merge semantics, and how γ behaves under merge, are open.

Complexity. Computing $\gamma(X)$ for an arbitrary X is a reverse reachability problem, linear in the size of the reference graph restricted to X . Cascade and dependents are a breadth-first search over two indexes. Effective validity adds one interval intersection per traversed reference. For the embedded scale targeted by the implementation (10^4 – 10^7 statements) these are cheap; at larger scales, materialising ν^* incrementally on write is the natural extension.

Limitations. The theorems are about the data model and its operations, not about query languages. We have not formalised how SPARQL’s default graph or Cypher’s dual view of statements interact with γ . The conformance findings show that the implementation has not yet met every hypothesis, and the empirical evidence is the project’s test suite, not an independent benchmark.

10 Conclusion

Statement identity, two clocks and a rule against forgetting are each well understood. Put together at the granularity of a single statement, they raise one new question: is what we see *grounded*? A single interior operator answers it. It explains the cascade as forced and minimal, characterises grounded snapshots by nested lifetimes, gives valid-time slices a closed-form effective validity, and corrects the replay of a correction. Metagraphs then follow as tags. Edges are already vertices, membership statements make any atom a container, and the Bitemporal Lifting Theorem lets every temporal operation act on metagraphs without a temporal theory of their own. The theory paid for itself during writing: it predicted three ungrounded states in a working implementation, and each correction it suggested is again an instance of γ .

Availability. Tiramemsu is open source under the MIT and Apache-2.0 licences. This preprint is distributed under CC BY 4.0.

References

- [1] Carlos E. Alchourrón, Peter Gärdenfors, and David Makinson. On the logic of theory change: Partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2): 510–530, 1985. doi: 10.2307/2274239.
- [2] Renzo Angles, Aidan Hogan, Ora Lassila, Carlos Rojas, Daniel Schwabe, Pedro Szekely, and Domagoj Vrgoč. Multilayer graphs: A unified data model for graph databases. In *Proceedings of the 5th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 1–6, 2022. doi: 10.1145/3534540.3534696.
- [3] Amit Basu and Robert W. Blanning. *Metagraphs and Their Applications*, volume 15 of *Integrated Series in Information Systems*. Springer, New York, 2007. ISBN 978-0-387-37233-4. doi: 10.1007/978-0-387-37234-1.
- [4] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001. doi: 10.1007/3-540-44503-X_20.
- [5] Jaewook Byun, Sungpil Woo, and Daeyoung Kim. ChronoGraph: Enabling temporal graph traversals for efficient information diffusion analysis over time. *IEEE Transactions on Knowledge and Data Engineering*, 32(3):424–437, 2020. doi: 10.1109/TKDE.2019.2891565.
- [6] Jeremy J. Carroll, Christian Bizer, Pat Hayes, and Patrick Stickler. Named graphs, provenance and trust. In *Proceedings of the 14th International Conference on World Wide Web (WWW)*, pages 613–622, 2005. doi: 10.1145/1060745.1060835.
- [7] Arnaud Casteigts, Paola Flocchini, Walter Quattrociocchi, and Nicola Santoro. Time-varying graphs and dynamic networks. *International Journal of Parallel, Emergent and Distributed Systems*, 27(5):387–408, 2012. doi: 10.1080/17445760.2012.668546. Conference version: ADHOC-NOW 2011, LNCS, pp. 346–359, doi:10.1007/978-3-642-22450-8_27.
- [8] Valeriy Chernenkiy, Yuriy Gapanyuk, Anatoly Nardid, Maria Skvortsova, Anton Gushcha, Yuriy Fedorenko, and Richard Picking. Using the metagraph approach for addressing RDF knowledge representation limitations. In *2017 Internet Technologies and Applications (ITA)*, pages 47–52. IEEE, 2017. doi: 10.1109/ITECHA.2017.8101909.

- [9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025.
- [10] Datomic. Datomic documentation: Data model. <https://docs.datomic.com/whatis/data-model.html>, 2026. Accessed 2026-10-03.
- [11] Johan de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986. doi: 10.1016/0004-3702(86)90080-9.
- [12] Ariel Debrouvier, Eliseo Parodi, Matías Perazzo, Valeria Soliani, and Alejandro Vaisman. A model and query language for temporal graph databases. *The VLDB Journal*, 30(5): 825–858, 2021. doi: 10.1007/s00778-021-00675-4.
- [13] Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979. doi: 10.1016/0004-3702(79)90008-0.
- [14] Johannes Frey, Kay Müller, Sebastian Hellmann, Erhard Rahm, and Maria-Esther Vidal. Evaluation of metadata representations in RDF stores. *Semantic Web*, 10(2):205–229, 2019. doi: 10.3233/SW-180307.
- [15] Yuriy Gapanyuk. The development of the metagraph data and knowledge model. In Valery B. Tarassov, Vadim V. Borisov, and Boris A. Kobrinskii, editors, *Russian Advances in Fuzzy Systems and Soft Computing: Selected Contributions to the 10th International Conference on Integrated Models and Soft Computing in Artificial Intelligence (IMSC-2021)*, volume 2965 of *CEUR Workshop Proceedings*, pages 1–7, 2021. URL <https://ceur-ws.org/Vol-2965/paper01.pdf>.
- [16] Ewout Gelling, George Fletcher, and Michael Schmidt. Statement graphs: Unifying the graph data model landscape. In *Database Systems for Advanced Applications (DASFAA 2024)*, Lecture Notes in Computer Science, pages 364–376. Springer, 2024. doi: 10.1007/978-981-97-5575-2_27.
- [17] Ben Goertzel. Folding and unfolding on metagraphs, 2020.
- [18] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 31–40, 2007. doi: 10.1145/1265530.1265535.
- [19] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 59532–59569, 2024. doi: 10.52202/079017-1902.
- [20] Claudio Gutierrez, Carlos A. Hurtado, and Alejandro Vaisman. Introducing time into RDF. *IEEE Transactions on Knowledge and Data Engineering*, 19(2):207–218, 2007. doi: 10.1109/TKDE.2007.34.
- [21] Olaf Hartig. Foundations of RDF* and SPARQL* (an alternative approach to statement-level metadata in RDF). In Juan Reutter and Divesh Srivastava, editors, *Proceedings of the 11th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW)*, volume 1912 of *CEUR Workshop Proceedings*, 2017. URL <https://ceur-ws.org/Vol-1912/paper12.pdf>.
- [22] Olaf Hartig, Pierre-Antoine Champin, Andy Seaborne, and Gregg Kellogg. RDF 1.2 concepts and abstract data model. W3C Candidate Recommendation Snapshot, April 2026. URL <https://www.w3.org/TR/2026/CR-rdf12-concepts-20260407/>. 7 April 2026.

- [23] Daniel Hernández, Aidan Hogan, and Markus Krötzsch. Reifying RDF: What works well with Wikidata? In *Proceedings of the 11th International Workshop on Scalable Semantic Web Knowledge Base Systems (SSWS)*, volume 1457 of *CEUR Workshop Proceedings*, pages 32–47, 2015. URL https://ceur-ws.org/Vol-1457/SSWS2015_paper3.pdf.
- [24] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):71:1–71:37, 2021. doi: 10.1145/3447772.
- [25] Petter Holme and Jari Saramäki. Temporal networks. *Physics Reports*, 519(3):97–125, 2012. doi: 10.1016/j.physrep.2012.03.001.
- [26] Jiamin Hou, Zhanhao Zhao, Zhouyu Wang, Wei Lu, Guodong Jin, Dong Wen, and Xiaoyong Du. AeonG: An efficient built-in temporal support in graph databases. *Proceedings of the VLDB Endowment*, 17(6):1515–1527, 2024. doi: 10.14778/3648160.3648187.
- [27] Borislav Iordanov. HyperGraphDB: A generalized graph database. In *Web-Age Information Management (WAIM 2010 Workshops)*, volume 6185 of *Lecture Notes in Computer Science*, pages 25–36. Springer, 2010. doi: 10.1007/978-3-642-16720-1_3.
- [28] Christian S. Jensen, Michael D. Soo, and Richard T. Snodgrass. Unifying temporal data models via a conceptual model. *Information Systems*, 19(7):513–547, 1994. doi: 10.1016/0306-4379(94)90013-2.
- [29] Christian S. Jensen, Curtis E. Dyreson, Michael Böhlen, James Clifford, Ramez Elmasri, Shashi K. Gadia, Fabio Grandi, Pat Hayes, Sushil Jajodia, Wolfgang Käfer, Nick Kline, Nikos Lorentzos, Yannis Mitsopoulos, Angelo Montanari, Daniel Nonen, Elisa Peressi, Barbara Pernici, John F. Roddick, Nandlal L. Sarda, Maria Rita Scalas, Arie Segev, Richard T. Snodgrass, Michael D. Soo, Abdullah Tansel, Paolo Tiberio, and Gio Wiederhold. The consensus glossary of temporal database concepts – February 1998 version. In Opher Etzion, Sushil Jajodia, and Suryanarayana Sripada, editors, *Temporal Databases: Research and Practice*, volume 1399 of *Lecture Notes in Computer Science*, pages 367–405. Springer, 1998. doi: 10.1007/BFb0053710.
- [30] David Kempe, Jon Kleinberg, and Amit Kumar. Connectivity and inference problems for temporal networks. *Journal of Computer and System Sciences*, 64(4):820–842, 2002. doi: 10.1006/jcss.2002.1829.
- [31] Mikko Kivelä, Alex Arenas, Marc Barthélemy, James P. Gleeson, Yamir Moreno, and Mason A. Porter. Multilayer networks. *Journal of Complex Networks*, 2(3):203–271, 2014. doi: 10.1093/comnet/cnu016.
- [32] Krishna Kulkarni and Jan-Eike Michels. Temporal features in SQL:2011. *ACM SIGMOD Record*, 41(3):34–43, 2012. doi: 10.1145/2380776.2380786.
- [33] Ora Lassila, Michael Schmidt, Olaf Hartig, Brad Bebee, Dave Bechberger, Willem Broekema, Ankesh Khandelwal, Kelvin Lawrence, Carlos Manuel Lopez Enriquez, Ronak Sharda, and Bryan Thompson. The OneGraph vision: Challenges of breaking the graph model lock-in. *Semantic Web*, 14(1):125–134, 2023. doi: 10.3233/SW-223273.
- [34] Maria Massri, Zoltan Miklos, Philippe Raipin, and Pierre Meye. Clock-G: A temporal graph management system with space-efficient storage technique. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, pages 2263–2276, 2022. doi: 10.1109/ICDE53745.2022.00215.

- [35] Vera Zaychik Moffitt and Julia Stoyanovich. Temporal graph algebra. In *Proceedings of the 16th International Symposium on Database Programming Languages (DBPL)*, pages 10:1–10:12, 2017. doi: 10.1145/3122831.3122838.
- [36] Vinh Nguyen, Olivier Bodenreider, and Amit Sheth. Don’t like RDF reification? Making statements about statements using singleton property. In *Proceedings of the 23rd International Conference on World Wide Web (WWW)*, pages 759–770, 2014. doi: 10.1145/2566486.2567973.
- [37] Fabrizio Orlandi, Damien Graux, and Declan O’Sullivan. Benchmarking RDF metadata representations: Reification, singleton property and RDF*. In *2021 IEEE 15th International Conference on Semantic Computing (ICSC)*, pages 233–240, 2021. doi: 10.1109/ICSC50631.2021.00049.
- [38] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.
- [39] Volodymyr Pavlyshyn. *Metagraph for AI Agents: Metagraphs for Agentic AI: Beyond Triples, Beyond Hypergraphs*. Leanpub, 2026. URL <https://leanpub.com/metagraphforaiagents>. Self-published e-book; last updated 24 March 2026.
- [40] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025.
- [41] Christopher Rost, Kevin Gomez, Matthias Täschner, Philip Fritzsche, Lucas Schons, Lukas Christ, Timo Adameit, Martin Junghanns, and Erhard Rahm. Distributed temporal graph analytics with GRADOOP. *The VLDB Journal*, 31(2):375–401, 2022. doi: 10.1007/s00778-021-00667-4.
- [42] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, San Francisco, 1999. ISBN 1-55860-436-7.
- [43] Richard T. Snodgrass and Ilsoo Ahn. A taxonomy of time in databases. In *Proceedings of the 1985 ACM SIGMOD International Conference on Management of Data*, pages 236–246. ACM, 1985. doi: 10.1145/318898.318921. Also in ACM SIGMOD Record 14(4), doi:10.1145/971699.318921.
- [44] Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5(2):285–309, 1955. doi: 10.2140/pjm.1955.5.285.
- [45] Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014. doi: 10.1145/2629489.
- [46] Domagoj Vrgoč, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. MillenniumDB: An open-source graph database system. *Data Intelligence*, 5(3):560–610, 2023. doi: 10.1162/dint_a_00229. Preprint: arXiv:2111.01540.
- [47] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory, 2024.
- [48] Huanhuan Wu, James Cheng, Silu Huang, Yiping Ke, Yi Lu, and Yanyan Xu. Path problems in temporal graphs. *Proceedings of the VLDB Endowment*, 7(9):721–732, 2014. doi: 10.14778/2732939.2732945.
- [49] XTDB. XTDB documentation: Time in XTDB. <https://docs.xtdb.com/about/time-in-xtdb.html>, 2026. Accessed 2026-10-03.

- [50] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents, 2025.